

Practica de Laboratorio 3 de Jesús Rachadell 31421494

1. Memory-mapped I/O e implicaciones de lw y sw

La memoria no se ve como una única unidad de Ram sino como un espacio de direccionamiento unificado, donde el hardware reserva un rango específico de direcciones para poder comunicarse con los periféricos o cualquier pieza de hardware externo. Normalmente se suelen reservar las direcciones más altas de la memoria, en MIPS32 eso es a partir de 0xFFFF0000 en adelante. lw y sw cumplen con el papel de leer y guardar respectivamente en estos espacios de memoria que están siendo afectados en tiempo real por hardware externo.

2. MMIO vs E/S

	MMIO	E/S
Instrucciones de manejo	Usa la memoria estándar lw y sw	Requiere instrucciones dedicadas
Ventajas	Diseño menos complejo	No utiliza espacio de la RAM
Desventaja	Consume espacio dentro de la memoria RAM	Requiere hardware adicional decodificar las señales

Cuadro 1: Diferencias

MIPS32 utiliza el MMIO principalmente para mantener la estructura RISC, evitando añadir instrucciones complejas de E/S

3. Solapamiento de direcciones

Si dos dispositivos utilizan la misma dirección ocurre una solapación, haciendo que ambos dispositivos respondan al mismo tiempo, esto puede generar problemas eléctricos o de por sí corromper el dato. La forma de lidiar con esto es con un Decodificador de direcciones, el cuál es un componente lógico que lo que hace es asegurar que solo uno se pueda activar a la vez.

4. Sobre el diseño con MMIO

MMIO simplifica el diseño porque este método no necesita saber si se está hablando con una memoria u otro dispositivo solo sabe que está accediendo a una dirección en la memoria. Para implementar E/S por puertos se necesitarían indicar si se está comunicando con la memoria o algún periférico, además de instrucciones específicas para manejar esos puertos.

5. Bus de datos y direcciones

Cuando se procesa una dirección esta se pasa por el bus de direcciones es aquí donde el Decodificador de Direcciones toma la decisión dependiendo de si la dirección pertenece a la RAM o el espacio reservado para periféricos, cuando pasa esto el periférico al que le pertenece está dirección toma el dato del bus de datos (en caso de sw) o lo envía al bus de datos (en caso de lw).

6. Acceso a la memoria

En la actualidad ningún programa hecho para el usuario puede acceder directamente a estas direcciones, al intentar hacerlo dará un error. Para asegurar esto se aplican mecanismos de Protección de memoria, permitiendo solo al sistema operativo en uso acceder a esas direcciones.

7. Evitar esperas activas

Una de las principales maneras de evitar las esperas activas propia de las prácticas es usar un sistema de interrupciones que lo que hace es dejar al CPU libre de hacer las tareas que necesita hacer, y una vez haya una señal por parte del periférico este manda una request para interrumpir al procesador el cual pausa lo que está haciendo y hace la tarea requerida para luego volver a lo que estaba haciendo.

8. Análisis y Discusión de los resultados

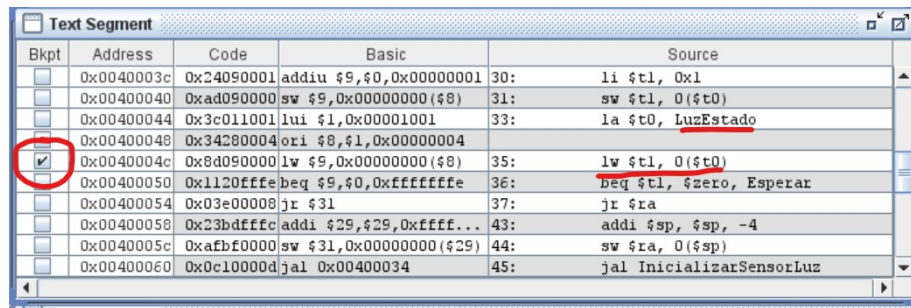
Procedo a mostrar el comportamiento y los distintos resultados dependiendo de la intervención.

8.1. Caso sin intervención

Para empezar, por la naturaleza de los ejercicios de esperar una señal por parte del “periférico” en cuestión, todos y cada uno de los ejercicios, si se dejan sin intervención se quedan ciclando esperando que su respectivo registro de estado dé la señal de que los datos están listos, se hace de esta manera para que si en algún punto de la iteración el periférico modifica el valor de la dirección el programa puede leerlo y procesarlo en tiempo real. Esto sería útil utilizando un sensor real pero como lo estamos simulando se necesita la intervención del usuario para que funcione.

8.2. Pruebas en tiempo real

Para poder demostrar que el algoritmo funciona lo primero es establecer el uso de breakpoints para la corrida, esto ayudará a poder intervenir en los momentos que se vaya a leer el valor de la dirección de los registros. por ejemplo:



Bkpt	Address	Code	Basic	Source
☐	0x0040003c	0x24090001	addiu \$9,\$0,0x00000001	30: li \$t1, 0x1
☐	0x00400040	0xad090000	sw \$9,0x00000000(\$8)	31: sw \$t1, 0(\$t0)
☐	0x00400044	0x3c011001	lui \$1,0x00001001	33: la \$t0, LuzEstado
☐	0x00400048	0x34280004	ori \$8,\$1,0x00000004	
☒	0x0040004c	0x8d090000	lw \$9,0x00000000(\$8)	35: lw \$t1, 0(\$t0)
☐	0x00400050	0x1120ffff	beq \$9,\$0,0xffffffff	36: beq \$t1, \$zero, Esperar
☐	0x00400054	0x03e00008	jr \$31	37: jr \$ra
☐	0x00400058	0x23bdfffc	addi \$29,\$29,0xffff...	43: addi \$sp, \$sp, -4
☐	0x0040005c	0xafbf0000	sw \$31,0x00000000(\$29)	44: sw \$ra, 0(\$sp)
☐	0x00400060	0x0c10000d	jal 0x00400034	45: jal InicializarSensorLuz

Figura 1: Uso de breakpoints

Una vez establecido eso, procedemos a iniciar el algoritmo, en este caso se usará el primero pero lo mismo aplica para los otros dos.

Cuando se da a correr el programa, lo primero que pasa es que frena el breakpoint que establecimos que en este caso es cuando va a leer el valor de la dirección de LuzEstado, en ese punto el sensor ya se inicializó, queda entonces que LuzEstado sea diferente a 0 para salir del bucle, aquí entonces se cambia el valor de LuzEstado manualmente para no quedarnos estancados.

Text Segment					
Bkpt	Address	Code	Basic	Source	
☐	0x0040003c	0x24090001	addiu \$9,\$0,0x00000001	30:	li \$t1, 0x1
☐	0x00400040	0xad090000	sw \$9,0x00000000(\$8)	31:	sw \$t1, 0(\$t0)
☐	0x00400044	0x3c011001	lui \$1,0x00001001	33:	la \$t0, LuzEstado
☐	0x00400048	0x34280004	ori \$8,\$1,0x00000004		
☒	0x0040004c	0x8d090000	lw \$9,0x00000000(\$8)	35:	lw \$t1, 0(\$t0)
☐	0x00400050	0x1120ffff	beq \$9,\$0,0xffffffff	36:	beq \$t1, \$zero, Esperar
☐	0x00400054	0x03e00008	jr \$31	37:	jr \$ra
☐	0x00400058	0x23bdfcfc	addi \$29,\$29,0xffff...	43:	addi \$sp, \$sp, -4
☐	0x0040005c	0xafbf0000	sw \$31,0x00000000(\$29)	44:	sw \$ra, 0(\$sp)
☐	0x00400060	0x0c10000d	jal 0x00400034	45:	jal InicializarSensorLuz

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000000	0x00000000	0x6f627548	0x206e7520	0x6f7272

Figura 2: Sin cambiar

Text Segment					
Bkpt	Address	Code	Basic	Source	
☐	0x0040003c	0x24090001	addiu \$9,\$0,0x00000001	30:	li \$t1, 0x1
☐	0x00400040	0xad090000	sw \$9,0x00000000(\$8)	31:	sw \$t1, 0(\$t0)
☐	0x00400044	0x3c011001	lui \$1,0x00001001	33:	la \$t0, LuzEstado
☐	0x00400048	0x34280004	ori \$8,\$1,0x00000004		
☒	0x0040004c	0x8d090000	lw \$9,0x00000000(\$8)	35:	lw \$t1, 0(\$t0)
☐	0x00400050	0x1120ffff	beq \$9,\$0,0xffffffff	36:	beq \$t1, \$zero, Esperar
☐	0x00400054	0x03e00008	jr \$31	37:	jr \$ra
☐	0x00400058	0x23bdfcfc	addi \$29,\$29,0xffff...	43:	addi \$sp, \$sp, -4
☐	0x0040005c	0xafbf0000	sw \$31,0x00000000(\$29)	44:	sw \$ra, 0(\$sp)
☐	0x00400060	0x0c10000d	jal 0x00400034	45:	jal InicializarSensorLuz

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000001	0x00000000	0x6f627548	0x206e7520	0x6f7272

Figura 3: Cambiamos LuzEstado

Luego de eso pueden pasar dos cosas, en caso de haya sido un error, entonces \$v0=-1 y \$v1=-1 indicando entonces que hubo en error y devuelta en main imprime el mensaje de que hubo un error en la lectura. La segunda cosa que puede pasar es que la lectura sea valida en ese caso ponemos un breakpoint cuando vayamos a leer los datos para poder modificar LuzDatos, esto se guarda en \$v0, y para finalizar \$v1=0, indicando que la lectura fue exitosa, devuelta en main imprime el número leído.

Text Segment					
Bkpt	Address	Code	Basic	Source	
☐	0x00400078	0x240affff	addiu \$10,\$0,0xffff...	53:	li \$t2, -1
☐	0x0040007c	0x112a0005	beq \$9,\$10,0x00000005	54:	beq \$t1, \$t2, Error #LuzEsta...
☐	0x00400080	0x3c011001	lui \$1,0x00001001	57:	la \$t0, LuzDatos
☐	0x00400084	0x34280008	ori \$8,\$1,0x00000008		
☒	0x00400088	0x8d020000	lw \$2,0x00000000(\$8)	58:	lw \$v0, 0(\$t0)
☐	0x0040008c	0x24030000	addiu \$3,\$0,0x00000000	59:	li \$v1, 0
☐	0x00400090	0x03e00008	jr \$31	60:	jr \$ra
☐	0x00400094	0x2402ffff	addiu \$2,\$0,0xffffffff	62:	li \$v0, -1
☐	0x00400098	0x2403ffff	addiu \$3,\$0,0xffffffff	63:	li \$v1, -1
☐	0x0040009c	0x03e00008	jr \$31	64:	jr \$ra

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000001	0x00000000	0x6f627548	0x206e7520	0x6f7272

Figura 4: LuzDatos sin cambios

Text Segment					
Bkpt	Address	Code	Basic	Source	
☐	0x00400078	0x240affff	addiu \$t0,\$0,0xffff...	53:	li \$t2, -1
☐	0x0040007c	0x112a0005	beq \$9,\$t0,0x00000005	54:	beq \$t1, \$t2, Error #LuzEsta...
☐	0x00400080	0x3c011001	lui \$t1,0x00001001	57:	la \$t0, LuzDatos
☐	0x00400084	0x34280008	ori \$8,\$t1,0x00000008		
☒	0x00400088	0x8d020000	lw \$2,0x00000000(\$8)	58:	lw \$v0, 0(\$t0)
☐	0x0040008c	0x24030000	addiu \$3,\$0,0x00000000	59:	li \$v1, 0
☐	0x00400090	0x03e00008	jr \$31	60:	jr \$ra
☐	0x00400094	0x2402ffff	addiu \$2,\$0,0xffffffff	62:	li \$v0, -1
☐	0x00400098	0x2403ffff	addiu \$3,\$0,0xffffffff	63:	li \$v1, -1
☐	0x0040009c	0x03e00008	jr \$31	64:	jr \$ra

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000001	0x00000030	0x6f627548	0x206e7520	0x6f7272

Figura 5: LuzDatos después

Como se mostró antes esto genera dos resultados posibles aparte de quedarse ciclando infinitamente:

```

48
-- program is finished running --

```

Figura 6: Caso exitoso

```

Hubo un error en la lectura
-- program is finished running --

```

Figura 7: En caso de error

8.3. Conclusión

Para los otros dos ejercicios la ejecución cambia pero en esencia se mantiene igual: inicializar hardware -¿Esperar que el registro de Estado envíe algo -¿En caso de error mostrarlo y en caso de éxito leer el dato. Estas técnicas nos son útiles para manejar información provenientes de periféricos, pese que en este caso se “simule” la entrada para probar la efectividad del algoritmo